Newt

A MacOS OpenXR/Vulkan Runtime Core

Triscuitcircuit

October 21, 2026

Contents

1 Newt **2**

2 About **2**
2.1 Capabilities 2

3 Installation **2**
3.1 Requirements (from source build) 3
3.2 Building and Installation 3
3.2.1 Optional Commands 3
3.2.2 Linking with CMAKE 4
3.3 Configuring the runtime (one time) 4

4 Embedding Newt **4**
4.1 The C++ API 4

5 Design **5**
5.1 Known Issues 5

6 Testing **5**

7 Contributing **6**

8 License **6**
8.1 Libraries used 6

Well she turned me into a newt. A newt? ... I got better.

– Monty Pyhton and the Holy Grail

1 Newt

2 About

`Newt` is a runtime wrapper for OpenXR, MoltenVK and Monado rendering under the MIT license. It is named after the Apple Newton, a device that was ahead of its time in many ways.

Also, a Newt is *not* a lizard, and this wrapper makes disconnecting from a certain walled garden just a bit easier.

Newt is split into two systems:

- `Newt` ((`libnewt_core.a`): A runtime library for communicating with Monando, OpenXR and MoltenVK
- `newtd`: A session-manager daemon for transfering to game engines
  - (A few Godot examples are included in this repository, which include usage of the `Newt` bridge addon)

2.1 Capabilities

- OpenXR session lifecycle, controllers, and haptic output via the `Newt::Session` C++ API.
- GPU (IOSurface) texture import to Vulkan, with media capture built on top of it.
- The daemon, `Newtd`, handles one-owner headset supervision.
- An optional generic libusb module (`NEWT_WITH_USB=ON`).
- Initiate headset pairing on macOS.

3 Installation

Building Newt and configuring its headset runtime are separate steps. The OpenXR SDK supplies the *loader*. The headset backend supplies a *runtime*. Installing the loader does not create Newt's runtime configuration.

3.1 Requirements (from source build)

- MacOS, with Xcode Command line tools (`xcode-select -install`)
- CMAKE 3.20+ (CMAKE.org or by Homebrew)
- The Vulkan SDK for MacOS, including `glslangValidator`
- An installed Khronos OpenXR SDK and loader, with its CMake package
- *Optional:* libusb + pkg-config, detected automatically (`brew install libusb pkg-config`)
- A compatible external OpenXR runtime (Monando)
- Python (for automatic tests)

  [!NOTE] Do not set `VULKAN_SDK` to `libMoltenVK.dylib`. Newt needs the SDK's `macOS` directory (headers, libraries, shader tools). Also, Newt's setup expects a runtime JSON manifest instead of just `libopenxr_loader.dylib`.

3.2 Building and Installation

The CMAKE build process is designed to find the SDKs. Specifically, it finds Vulkan with `/VulkanSDK` and OpenXR SDK under your `brew` install path. (`/opt/homebrew` or `/usr/local`).

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=/your/install/p
DBUILD_TESTING=ON
cmake --build build
ctest --test-dir build --output-on-failure
cmake --install build
```

This builds a static library of Newt (`libnewt_core.a`) and the native `newtd` session-management Command Line Interface

3.2.1 Optional Commands

- `NEWT_WITH_USB=AUTO` for building with Libusb.
- `NEWT_BUILD_DAEMON=OFF` to skip building `newtd` entirely.

3.2.2 Linking with CMAKE

If you want to link against Newt with CMAKE, you can use `find_package(Newt CONFIG REQUIRED)` with `Newt::core`. .

If OpenXR isn't under one of those curated prefixes, you can direct the build with: `-DOpenXR_DIR=/path/to/folder/containing/OpenXRConfig.cmake` for one folder, or `-DNEWT_SDK_SEARCH_PATH=/some/prefix` to search a whole path. You can also use the included Python code to find likely install locations.

```
python3 tools/find-libraries.py --root /path/to/sdk-sources
```

3.3 Configuring the runtime (one time)

Set `XR_RUNTIME_JSON` to the runtime manifest and configure the Vulkan loader (if the runtime requires it).

```
newt configure --runtime /path/to/runtime.json \
               --icd /path/to/MoltenVK_icd.json
# or, for a host provided dashboard executable (like PippinVR):
newt configure --dashboard /path/to/host-dashboard \
               --runtime /path/to/runtime.json \
               --icd /path/to/MoltenVK_icd.json
```

This can also be done when installing with CMAKE by passing `-DNEWT_CONFIGURE_RUNTIME=/path/to/` and the `cmake -install` runs the equivalent `newt configure`. Configuration is stored in `~~/Library/Application Support/Newt/config.json`. From a source checkout, use the built daemon (`build/newtd`, installed as `newt`) in place of the installed `newt` command.

In Godot, the same paths can be picked with **Project -> Tools -> Configure Newt**.

4 Embedding Newt

4.1 The C++ API

As mentioned you can link `Newt::core`, and include `<Newt/Session.h>` in your code. `Newt::Session::Run(delegate)` owns the initialization process input polling, frame submission, and cleanup, synchronously, on the calling thread.

`RequestStop()` may be called from another thread. The delegate hooks are:

- `OnSystem` (host policy checks)
- `OnReady` (allocate host resources)
- `OnInput`, `OnPrepareFrame` (upload content)
- `OnDrawEye` (draw inside an active render pass)
- `OnStopping` (release resources before the device is destroyed).

Logging can be routed to the host through `include/NewtLog.h`. However, if no logging handler is installed, the output will go to stdout.

5 Design

Newt is split into a product-neutral runtime core and into different layers off the core.

Component	Responsibility
Core (`newt_core`, `Newt::core`)	OpenXR session lifecycle, tracking/controller i
Session manager (`daemon/newtd.mm`, binary `newtd`)	One-owner supervision of a game and a host's
External runtime (e.g. Monado + MoltenVK)	Headset USB transport and graphics translatio

OpenXR swapchain images are borrowed and never destroyed by Newt.

5.1 Known Issues

- Every dashboard to game handoff (and vice versa) requires different USB pairing headset requests.
- Headset USB audio is not implemented.
- A Godot game launched through the manager loads two independent MoltenVK copies into one proces.

6 Testing

Build with `BUILD_TESTING=ON` (the installer does), then:

```
ctest --test-dir build --output-on-failure
python3 tests/session_manager_test.py
```

The suites cover session lifecycle, the session-manager protocol against fake children (including the no-dashboard-configured path), optional-USB error cleanup, and a core-boundary check (fresh install, consumed by a plain C++ program with no product UI anywhere in the tree). None of them start VR, open a camera, or claim a USB device. The manager suite needs permission to create local Unix sockets.

7 Contributing

Contributions are welcome, and keep Newt free of vendored binaries see `DEPENDENCIES.md`.

This project will take contributions in the form of pull-requests.

8 License

MIT License. See `LICENSE`.

8.1 Libraries used

- OpenXR SDK/loader (Apache-2.0/MIT)
- Vulkan/MoltenVK (Apache-2.0)
- (optional and dynamically linked) libusb (LGPL-2.1-or-later)